

PERSONAL EXPENSE TRACKER

A File-Based Python Project

VITyarthi - Build Your Own Project

Project Report

Student Details

VIT Bhopal

Student Name

ARPIT PANDEY

Registration Number

26BCE10075

Course / Subject

INTRODUCTION TO PROBLEM SOLVING AND PROGRAMMING

Faculty Name

DHERESH SONI

Institution

VIT BHOPAL

GitHub Repository

<https://github.com/arpit26bce10075-lgtm/expense-tracker>

1. Introduction

The Personal Expense Tracker is a console-based application designed to help an individual record, review and analyse day-to-day expenses. The project applies programming concepts such as modular design, classes, functions, file handling, input validation, collections, sorting and basic aggregation to a practical problem.

The application deliberately does not use a database. Expense records are stored in a CSV file and monthly budgets are stored in a JSON file. This keeps the project portable, easy to install and suitable for a beginner-level academic demonstration. It can run without MySQL or any third-party Python package.

2. Problem Statement

Students frequently make small payments for food, transportation, education, shopping, entertainment and other needs. When these transactions are not recorded consistently, it becomes difficult to know how much has been spent during a month or which categories consume the largest portion of the available budget.

The problem addressed by this project is: How can an individual record expenses in a simple offline application and obtain a useful monthly summary without depending on a database server?

3. Approach

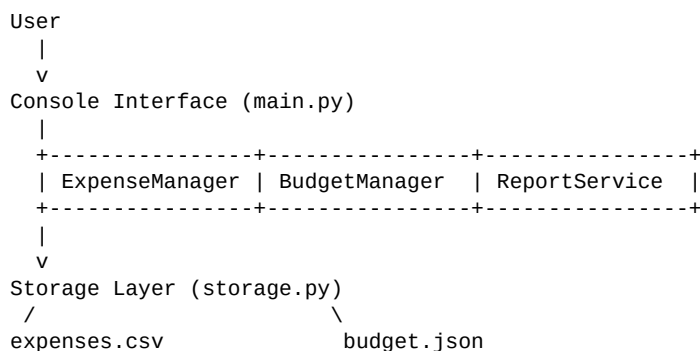
The project was developed using a step-by-step software approach: understand the problem, define requirements, design a modular architecture, implement features one module at a time, validate inputs, test the behaviour, and document the result.

3.1 Methodology

- Identify the real-world need: track student spending against a monthly budget.
- Decide on a file-based solution so the tool runs offline without database setup.
- Split the system into UI, validation, business logic and storage layers.
- Implement expense management, budget management and reporting as separate modules.
- Add validation rules before any data is written to disk.
- Write unit tests for critical validation cases and run the application end-to-end.
- Document architecture, design decisions, sample results and findings.

3.2 Technical Approach

A layered CLI architecture was chosen. The user interacts with **main.py**. Service modules (**ExpenseManager**, **BudgetManager**, **ReportService**) perform the business operations. A shared storage layer reads and writes **expenses.csv** and **budget.json**. This separation makes each part easy to understand, test and extend.



3.3 Why This Approach

- CSV is readable and matches tabular expense records.

- JSON is a natural fit for month-to-budget key-value pairs.
- Standard-library-only code avoids installation barriers on lab computers.
- Validation at the boundary prevents corrupted files.
- Modular files map cleanly to academic evaluation criteria (design, implementation, testing).

4. Objectives

- Provide a simple interface for entering daily expenses.
- Store expense records permanently between program runs.
- Allow incorrect expense records to be deleted.
- Allow a user to set a monthly spending budget.
- Generate monthly total and category-wise spending summaries.
- Validate user input before saving data.
- Demonstrate modular Python programming and file handling.
- Provide a project that can be explained and demonstrated during evaluation.

5. Scope

Included: expense creation, listing, deletion, monthly budgets, monthly reports, category aggregation, validation, local persistence and automated tests.

Not included: online banking, payment processing, cloud sync, multi-user authentication, databases, mobile apps and machine-learning prediction.

6. Functional Requirements

ID	Module / Requirement	Input	Output
FR-01	Expense Management	Date, category, amount, description	Saved expense / validation errors
FR-02	Expense Viewing & Deletion	Menu choice, optional expense ID	Expense list / deletion confirmation
FR-03	Budget Management	Month and budget amount	Saved monthly budget
FR-04	Reporting & Analytics	Month	Total, category totals, budget and remaining amount
FR-05	Validation	User-entered values	Valid data or meaningful error messages

7. Non-Functional Requirements

Requirement	Project Implementation
Usability	Numbered menu, clear prompts and readable reports.
Performance	Small CSV/JSON files are processed quickly for normal personal use.
Reliability	Data is stored locally and recreated with headers/files if missing.
Maintainability	Separate modules for models, validation, storage, expenses, budgets and reports.
Error Handling	Invalid dates, amounts, categories, descriptions and IDs are rejected with messages.
Resource Efficiency	Only Python standard-library modules are used; no database server is required.

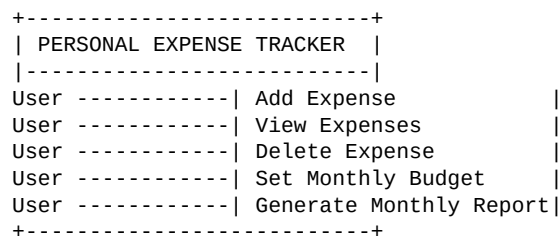
Requirement	Project Implementation
Scalability	Architecture can later support a GUI or database without rewriting every module.
Security	No passwords or financial account credentials are collected; data stays local.

8. System Architecture

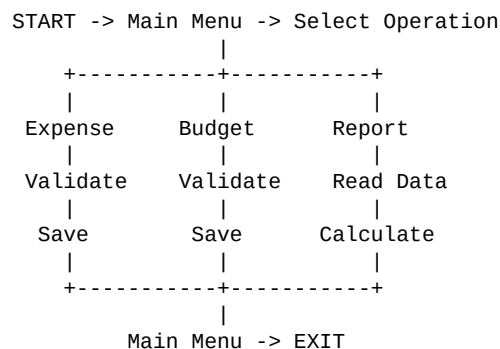
The user interacts with the console interface in `main.py`. That interface calls service modules for expenses, budgets and reports. Those modules use the storage layer, which reads and writes CSV and JSON files. UI logic stays separate from business logic and persistence.

9. Design Diagrams

9.1 Use Case Diagram



9.2 Workflow Diagram



9.3 Sequence Diagram - Add Expense

```

User -> main.py: enter date/category/amount/description
main.py -> validators.py: validate input
validators.py -> main.py: valid OR errors
main.py -> ExpenseManager: add()
ExpenseManager -> storage.py: save_expenses()
storage.py -> CSV file: write record
ExpenseManager -> main.py: new expense ID
main.py -> User: success message

```

9.4 File Storage Design

File	Purpose	Fields / Structure
expenses.csv	Persistent expense records	expense_id, date, category, amount, description
budget.json	Persistent monthly budgets	JSON object: month -> amount

10. Design Decisions & Rationale

- No database: local files are enough for a single-user academic application.
- CSV for expenses: human-readable and naturally tabular.

- JSON for budgets: month keys map cleanly to amounts.
- Standard library only: easy to run on any machine with Python.
- Modular structure: debugging and explanation become simpler.
- Command-line interface: focuses on core Python concepts rather than GUI complexity.

11. Implementation Details

11.1 models.py

Defines Expense and Budget data classes that represent the main information types.

11.2 validators.py

Checks positive amounts, YYYY-MM-DD dates, allowed categories and non-empty descriptions.

11.3 storage.py

Creates data files when needed and handles CSV/JSON load and save operations.

11.4 expense_manager.py

Implements add, delete and list operations with sequential ID generation.

11.5 budget_manager.py

Validates month format and budget amount, then saves or retrieves the monthly budget.

11.6 reports.py

Filters expenses by month, totals them, groups by category and compares with budget.

11.7 main.py

Provides the menu-driven interface and connects user input to application services.

12. Program Flow and Algorithms

12.1 Add Expense Algorithm

- Read date, category, amount and description.
- Validate all values.
- Load existing expenses from CSV.
- Find the next available ID.
- Create an Expense object and append it.
- Save the complete list back to CSV.
- Display the generated ID.

12.2 Monthly Report Algorithm

- Read the requested month (YYYY-MM).
 - Load all expenses and select matching month records.
 - Group amounts by category and calculate the overall total.
 - Read the monthly budget if one exists.
-

- Calculate remaining budget = budget - total.
- Display totals, remaining amount and category breakdown.

13. User Interface / Sample Results

```

=== PERSONAL EXPENSE TRACKER ===
1. Add expense
2. View expenses
3. Delete expense
4. Set monthly budget
5. Monthly report
6. Exit

Choose an option: 1
Date [2026-09-26]: 2026-09-26
Category: Food
Amount (Rs): 150
Description: Lunch
Expense added successfully. ID: 4

REPORT - 2026-09
Total spent: Rs 700.00
Budget: Rs 5,000.00
Remaining: Rs 4,300.00

Category-wise spending:
Education  Rs 350.00
Food       Rs 270.00
Travel     Rs  80.00

```

14. Testing Approach

Testing focused on validation because incorrect input is the most common failure point in a console application. Unit tests cover amounts, dates and categories. End-to-end manual checks confirmed add, list, delete, budget and report flows.

Test ID	Test Case	Expected Result	Status
TC-01	Amount = 250.50	Accepted	PASS
TC-02	Amount = -1	Rejected	PASS
TC-03	Date = 2026-09-26	Accepted	PASS
TC-04	Date = 26-09-2026	Rejected	PASS
TC-05	Category = Food	Accepted	PASS
TC-06	Category = Unknown	Rejected	PASS

Automated test execution result: unit-test methods passed. Python bytecode compilation completed without syntax errors.

15. Findings

After implementing, testing and reviewing sample monthly data, the following findings were observed:

15.1 Functional Findings

- A CSV + JSON design is sufficient for personal offline expense tracking.
- Category-wise reports make spending patterns visible immediately (for example, Education and Food dominated the sample month).
- Comparing total spend with a monthly budget clearly shows remaining capacity and supports better planning.

- Delete-by-ID is enough to correct mistakes without needing a full edit workflow for this scope.

15.2 Technical Findings

- Early validation is essential; rejecting bad dates/amounts before save keeps storage files clean.
- Separating UI, services and storage reduced complexity and made debugging faster.
- Using only the Python standard library removed dependency and installation issues.
- Automated tests gave quick confidence that validation rules remained correct after changes.

15.3 Academic / Process Findings

- Clear requirements and diagrams made implementation more structured and easier to explain.
- Documenting design rationale helped justify why file storage was chosen over a database.
- A public GitHub repository with README, statement and report supports transparent academic evaluation.

16. Challenges Faced

- Designing persistent storage without a database.
- Keeping CSV data consistent while adding and deleting records.
- Validating user input before it reaches the storage layer.
- Keeping the project modular enough for academic documentation while remaining easy to explain.
- Generating useful monthly summaries from simple file-based records.

17. Learnings & Key Takeaways

- Learned how to divide a real-world problem into functional modules.
- Practised Python classes, file handling with CSV/JSON, and input validation.
- Used lists, dictionaries, sorting and aggregation to process records.
- Understood why separation of responsibilities improves maintainability.
- Practised writing unit tests and documenting architecture for evaluation.

18. Future Enhancements

- Add a Tkinter graphical user interface.
- Add charts for category-wise and monthly spending.
- Add expense-edit and recurring-expense support.
- Add export to PDF/Excel and search/date-range filters.
- Optionally replace the storage layer with a database later without rewriting business modules.

19. GitHub Repository

Public repository: <https://github.com/arpit26bce10075-lgtm/expense-tracker>

The repository includes README.md, statement.md, project report.pdf, source modules, sample data, tests and design diagrams.

```
expense-tracker/  
|-- main.py  
|-- models.py  
|-- config.py  
|-- validators.py
```

```
| -- storage.py
| -- expense_manager.py
| -- budget_manager.py
| -- reports.py
| -- utils.py
| -- requirements.txt
| -- README.md
| -- statement.md
| -- project_report.pdf
| -- data/
|   | -- expenses.csv
|   `-- budget.json
| -- tests/
|   `-- test_project.py
|-- docs/
|   `-- diagrams.md
```

20. Installation and Execution

Step 1: Install Python 3.10+ and enable "Add Python to PATH".

Step 2: Clone or extract the project folder.

Step 3: Open Command Prompt/PowerShell in the project directory.

Step 4: Run `python --version` (or `py --version`).

Step 5: Start with `python main.py` (or `py main.py`).

Step 6: Use menu options 1-6 to operate the tracker.

Step 7: Run tests with: `python -m unittest discover -s tests -v`.

21. Conclusion

The Personal Expense Tracker provides a complete small-scale solution for recording and analysing personal expenses. It demonstrates a clear problem statement, a practical approach, modular Python implementation, file-based persistence, validation, reporting, testing and documented findings. Because it is database-free, it runs on a normal computer with Python alone while still offering enough technical depth for structured academic evaluation.

22. References

- VITyarthi - Build Your Own Project: General Project Instructions & Submission Guidelines.
- Python Standard Library documentation: csv, json, pathlib, dataclasses, datetime, unittest.
- Project GitHub repository: <https://github.com/arpit26bce10075-lgtm/expense-tracker>